React Week–1: Core Foundations — MAANG Interview Handbook

JSX

• Syntax extension for JavaScript, not HTML
• Used to describe UI declaratively
• Compiled by Babel to React.createElement(type, props, children)
• Supports expressions, not statements
• Escapes values by default to prevent XSS

MAANG Q: What does JSX compile to behind the scenes?
Trap: JSX is HTML inside JavaScript
Correct: JSX is syntactic sugar that compiles to React.createElement calls

Functional Components

• Plain JavaScript functions returning React elements
• Must be pure: same props always return same UI
• No side effects during render phase
• Hooks are usable only inside functional components

MAANG Q: Why must functional components be pure?
Trap: Purity is just a convention
Correct: Purity ensures predictable rendering and correct reconciliation

Props vs State

• Props are inputs passed from parent components
• Props are immutable inside the component
• State is managed locally using hooks like useState
• State updates trigger re-render
• State should live at the closest common owner

MAANG Q: How do you decide whether something should be props or state?
Trap: Anything that changes should be state everywhere
Correct: Decision is based on ownership and single source of truth

Controlled vs Uncontrolled Components

• Controlled components use React state as the source of truth
• Input value controlled via value and onChange
• Uncontrolled components let the DOM manage state
• Access values using refs
• Trade-off between predictability and performance

MAANG Q: Pros and cons of controlled vs uncontrolled inputs?
Trap: Uncontrolled components are bad practice
Correct: Controlled are predictable; uncontrolled are acceptable for simple or perf-critical cases

useEffect (Basics)

• Used to handle side effects like data fetching and subscriptions
• Runs after render phase
• Dependency array controls re-execution
• Cleanup function prevents memory leaks
• Commonly used for event listeners

MAANG Q: What causes useEffect to re-run?
Trap: It runs after every render always
Correct: It re-runs when values in the dependency array change

useEffect (Deep Dive)

• Effect callback must be synchronous
• Async functions return Promises, not cleanup functions
• Missing dependencies cause stale closures
• Updating a dependency inside effect can cause infinite loops
• Debounce patterns use setTimeout with cleanup

MAANG Q: Why can't useEffect callback be async?
Trap: React does not support async code
Correct: Async functions return Promises, but React expects a cleanup function or nothing

MAANG Q: What is a stale closure and how do you avoid it?
Trap: Stale closure only happens due to bugs
Correct: It happens when effects capture outdated state; fix with correct deps or functional updates

Rendering, Reconciliation & Keys

• React uses a Virtual DOM to compute UI changes
• Reconciliation diffs previous and next trees
• Only minimal DOM mutations are applied
• Keys give stable identity to list elements
• Using index as key breaks identity on reorder

MAANG Q: Why should you avoid using index as a key?
Trap: Index is fine because it's unique
Correct: Index breaks element identity and causes UI and state bugs

Mini Project: Movie Search

• Implements real-world async data flow
• Uses OMDb API for fetching data
• Must handle loading, error, and empty states
• Demonstrates component composition and effects

MAANG Q: How would you handle errors in this project?
Trap: Try-catch inside every component
Correct: Use error boundaries with fallback UI at feature or page level

Error Boundaries

• Catch errors during rendering and lifecycle methods
• Do not catch async or event handler errors
• Implemented using class components
• Used to prevent full app crashes

MAANG Q: Why can't hooks implement error boundaries?
Trap: Hooks are incomplete
Correct: React requires class lifecycle methods for error boundaries

MAANG Rapid Fire

• JSX is JavaScript, not HTML
• Props are immutable locally
• State updates always trigger re-render
• Effects synchronize external systems
• Cleanup prevents memory leaks
• Keys affect reconciliation correctness